

# OptiCrop: Smart Agricultural Production Optimization Engine

## Project Description:

The OptiCrop: Smart Agricultural Production Optimization Engine is a Machine Learning-based application developed to help farmers make better crop selection decisions. The system analyzes important soil and environmental parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall to recommend the most suitable crop.

The project aims to improve agricultural productivity by providing accurate, data-driven crop recommendations. It helps farmers optimize the use of natural resources, increase crop yield, and reduce the risk of selecting unsuitable crops. OptiCrop can also be used by agricultural researchers and policymakers to study crop-environment relationships and support sustainable farming practices. By using Machine Learning algorithms, the system enables faster and more reliable decision-making in agriculture.

The project includes data collection, data preprocessing, model training, crop prediction, and a user-friendly web application that allows users to enter soil and climate details and receive instant crop recommendations.

## Scenarios:

### Scenario 1: Smart Crop Recommendation for Farmers

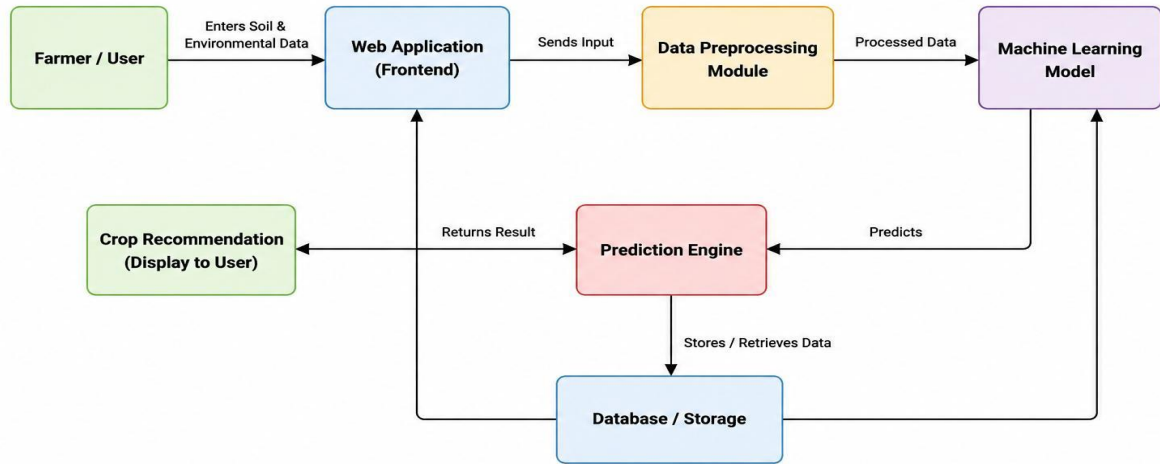
A farmer enters soil and environmental details such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall into the application. The Machine Learning model analyzes these parameters and recommends the most suitable crop for cultivation. This helps the farmer improve crop yield, reduce farming risks, and make better decisions based on data.

### Scenario 2: Crop Suitability and Environmental Assessment

A user wants to know whether the current soil and climate conditions are suitable for growing a particular crop. The system evaluates the provided soil and weather parameters and determines the crop's suitability. It also provides insights into expected productivity and environmental compatibility, helping users make informed agricultural decisions.

**Scenario 3: Agricultural Research and Policy Planning** Agricultural researchers and policymakers use the application to study the relationship between soil conditions, climate factors, and crop performance. By agricultural data and prediction results, they can identify farming patterns, support sustainable agriculture, and develop better policies for resource management and crop planning.

## Technical Architecture:

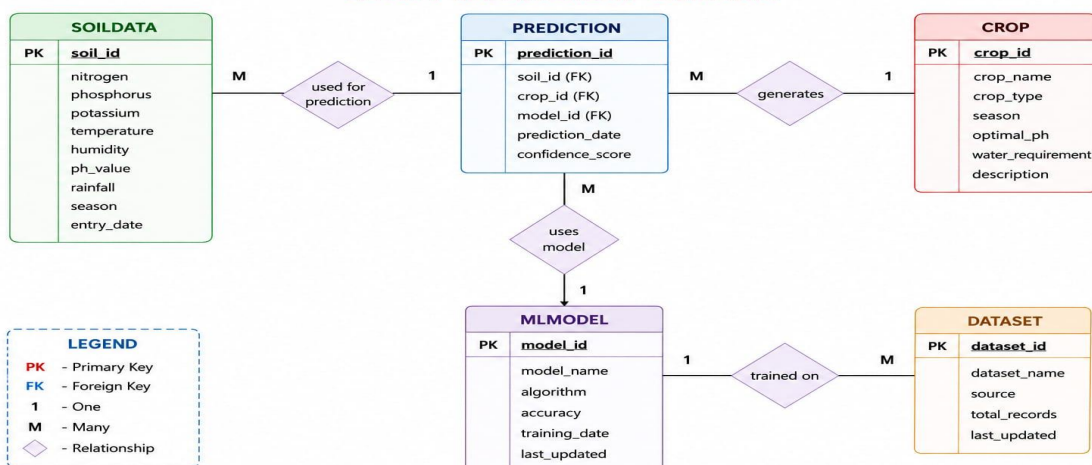


## Description:

The OptiCrop: Smart Agricultural Production Optimization Engine uses a simple Machine Learning architecture to recommend the most suitable crop based on soil and environmental conditions. The user enters soil parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall through the web application. The input data is pre-processed and sent to the Machine Learning model for analysis. The prediction engine generates the best crop recommendation and displays the result to the user. The system also stores the input and prediction results for future reference analysis.

## Entity Relation Diagram:

**OPTICROP: SMART AGRICULTURAL PRODUCTION OPTIMIZATION ENGINE**  
**ENTITY RELATIONSHIP DIAGRAM**



## Pre-requisites:

- **Anaconda Navigator:** <https://www.anaconda.com/download>
- **VS Code:** <https://code.visualstudio.com/>
- **NumPy:** <https://numpy.org/doc/stable/>
- **Pandas:** <https://pandas.pydata.org/docs/>
- **Scikit-learn:** <https://scikit-learn.org/stable/>
- **Matplotlib:** <https://matplotlib.org/stable/>
- **Seaborn:** <https://seaborn.pydata.org/>
- **Flask:** <https://flask.palletsprojects.com/>

## Project Workflow:

### Milestone 1: Project Planning

- **Activity 1.1:** Identify the challenges faced by farmers in selecting suitable crops based on soil and environmental conditions. Define the problem that the system aims to solve using Machine Learning.
- **Activity 1.2:** Gather the functional and non-functional requirements needed for the crop recommendation system. Define the project objectives, user requirements, and expected outcomes.
- **Activity 1.3:** Study existing research papers and crop recommendation systems based on Machine Learning. Analyze different approaches to identify suitable techniques for the project.
- **Activity 1.4:** Evaluate how the system improves agricultural productivity and supports sustainable farming. Analyze its benefits for farmers, agricultural organizations, and society.

### Milestone 2: Data Collection and Analysis

- **Activity 2.1:** Collect the crop recommendation dataset containing soil nutrients, weather conditions, and crop information. Verify the quality and completeness of the collected data.
- **Activity 2.2:** Import the required Python libraries such as Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn. Prepare the development environment for data analysis.
- **Activity 2.3:** Explore the dataset to understand its features, target variables, and data distribution. Identify relationships between different agricultural parameters.
- **Activity 2.4:** Perform Univariate Analysis to study the distribution of individual agricultural features using histogram plots.
- **Activity 2.5:** Perform Bivariate Analysis to analyze the relationship between crop labels and agricultural features using scatter plots.

- **Activity 2.6:** Perform Multivariate Analysis to examine the combined relationship among multiple agricultural features for better data understanding.

### **Milestone 3: Data Pre-processing**

- **Activity 3.1:** Detect missing values and remove duplicate records to improve data quality. Ensure the dataset is clean and consistent before model training.
- **Activity 3.2:** Identify abnormal values that may affect prediction accuracy. Apply suitable techniques to reduce the impact of outliers.
- **Activity 3.3:** Select the most relevant soil and environmental parameters for crop prediction. Remove unnecessary features to improve model efficiency.
- **Activity 3.4:** Divide the dataset into training and testing datasets. Prepare the data for Machine Learning model development and evaluation.

### **Milestone 4: Model Development**

- **Activity 4.1:** Apply K-Means Clustering to group similar agricultural conditions. Analyze clusters to understand hidden data patterns.
- **Activity 4.2:** Train the Logistic Regression model using the processed dataset. Use the model to predict the most suitable crop.
- **Activity 4.3:** Evaluate the trained model using appropriate performance metrics. Compare the prediction results to measure model accuracy.
- **Activity 4.4:** Save the trained Machine Learning model for future predictions. Prepare the model for integration with the web application.

### **Milestone 5: Application Development**

- **Activity 5.1:** Develop a responsive web interface using HTML, CSS, and Bootstrap. Design user-friendly pages for entering agricultural data.
- **Activity 5.2:** Develop the backend using Flask and connect it with the Machine Learning model. Process user requests and generate predictions.
- **Activity 5.3:** Test the complete application by entering different agricultural input values and verify that the system displays accurate crop recommendations.

### **Milestone 6: Conclusion**

## **Milestone 1: Project Planning:**

This milestone focuses on understanding the agricultural problem and defining the objectives of the crop recommendation system. It includes identifying farmers' challenges, gathering business requirements, reviewing existing research, and analyzing the social and business impact of the proposed solution. The knowledge gained during this phase forms the foundation for designing and developing an effective Machine Learning-based crop recommendation system.

### **Activity 1.1: Specify the Business Problem**

The objective of this activity is to identify the key challenges faced by farmers in selecting suitable crops based on soil nutrients and environmental conditions. The project studies factors such as Nitrogen (N), Phosphorous (P), Potassium (K), pH, temperature, humidity, and rainfall to understand their impact on crop growth.

The identified problem is solved using a Machine Learning-based crop recommendation system that analyzes agricultural data and predicts the most suitable crop. This helps improve agricultural productivity, reduce crop failure, optimize resource utilization, and support sustainable farming practices.

### **Activity 1.2: Business Requirement**

The objective of this activity is to identify and define the functional and non-functional requirements of the crop recommendation system. The application is designed to analyze agricultural and environmental parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), pH, temperature, humidity, and rainfall to recommend the most suitable crop for cultivation.

The system should provide a simple and user-friendly interface where farmers can enter agricultural data and receive accurate crop recommendations in real time. It should integrate Machine Learning algorithms with the web application to ensure reliable predictions, efficient resource utilization, and improved agricultural productivity while supporting sustainable farming practices.

### **Activity 1.3: Literature Survey**

The objective of this activity is to study existing research papers, agricultural recommendation systems, and Machine Learning techniques used for crop prediction. Various journals, online resources, and research articles are reviewed to understand current approaches, their advantages, limitations, and practical applications in agriculture.

The survey includes studying algorithms such as K-Nearest Neighbors (KNN), Decision Tree, Random Forest, Logistic Regression, K-Means Clustering, and Neural Networks. The knowledge gained from this literature survey helps identify suitable methodologies.

## **Activity 1.4: Social and Business Impact**

The objective of this activity is to evaluate the social and business benefits of implementing a Machine Learning-based crop recommendation system. The project aims to improve agricultural productivity by helping farmers select suitable crops based on soil nutrients and environmental conditions, reducing crop failure and increasing profitability.

The proposed solution promotes sustainable farming by optimizing the use of water, fertilizers, and other agricultural resources. It also supports agribusiness organizations and policymakers by providing reliable, data-driven insights for better planning and decision-making. Overall, the system contributes to food security, environmental sustainability, and the adoption of smart farming technologies.

## Milestone 2: Data Collection and Analysis:

The crop recommendation dataset was collected and analyzed to understand its features and data distribution. The dataset includes soil nutrients, temperature, humidity, pH, rainfall, and crop labels. Initial analysis was performed to examine the dataset structure, data types, and statistical information before preprocessing and model building.

### Activity 2.1: Download the Dataset

The first step in this phase is to collect a reliable agricultural dataset for developing the crop recommendation system. The **Crop\_recommendation.csv dataset** was downloaded from Kaggle for developing the crop recommendation system. It contains important features such as Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, Rainfall and Crop Label. The downloaded dataset, shown below, is used for training and testing the Machine Learning models.

|    | A  | B  | C  | D           | E           | F           | G           | H     |
|----|----|----|----|-------------|-------------|-------------|-------------|-------|
| 1  | N  | P  | K  | temperature | humidity    | ph          | rainfall    | label |
| 2  | 90 | 42 | 43 | 20.87974371 | 82.00274423 | 6.502985292 | 202.9355362 | rice  |
| 3  | 85 | 58 | 41 | 21.77046169 | 80.31964408 | 7.038096361 | 226.6555374 | rice  |
| 4  | 60 | 55 | 44 | 23.00445915 | 82.3207629  | 7.840207144 | 263.9642476 | rice  |
| 5  | 74 | 35 | 40 | 26.49109635 | 80.15836264 | 6.980400905 | 242.8640342 | rice  |
| 6  | 78 | 42 | 42 | 20.13017482 | 81.60487287 | 7.628472891 | 262.7173405 | rice  |
| 7  | 69 | 37 | 42 | 23.05804872 | 83.37011772 | 7.073453503 | 251.0549998 | rice  |
| 8  | 69 | 55 | 38 | 22.70883798 | 82.63941394 | 5.70080568  | 271.3248604 | rice  |
| 9  | 94 | 53 | 40 | 20.27774362 | 82.89408619 | 5.718627178 | 241.9741949 | rice  |
| 10 | 89 | 54 | 38 | 24.51588066 | 83.5352163  | 6.685346424 | 230.4462359 | rice  |
| 11 | 68 | 58 | 38 | 23.22397386 | 83.03322691 | 6.336253525 | 221.2091958 | rice  |
| 12 | 91 | 53 | 40 | 26.52723513 | 81.41753846 | 5.386167788 | 264.6148697 | rice  |
| 13 | 90 | 46 | 42 | 23.97898217 | 81.45061596 | 7.50283396  | 250.0832336 | rice  |
| 14 | 78 | 58 | 44 | 26.80079604 | 80.88684822 | 5.108681786 | 284.4364567 | rice  |
| 15 | 93 | 56 | 36 | 24.01497622 | 82.05687182 | 6.98435366  | 185.2773389 | rice  |
| 16 | 94 | 50 | 37 | 25.66585205 | 80.66385045 | 6.94801983  | 209.5869708 | rice  |
| 17 | 60 | 48 | 39 | 24.28209415 | 80.30025587 | 7.042299069 | 231.0863347 | rice  |
| 18 | 85 | 38 | 41 | 21.58711777 | 82.7883708  | 6.249050656 | 276.6552459 | rice  |
| 19 | 91 | 35 | 39 | 23.79391957 | 80.41817957 | 6.970859754 | 206.2611855 | rice  |
| 20 | 77 | 38 | 36 | 21.8652524  | 80.1923008  | 5.953933276 | 224.5550169 | rice  |
| 21 | 88 | 35 | 40 | 23.57943626 | 83.58760316 | 5.85393208  | 291.2986618 | rice  |
| 22 | 89 | 45 | 36 | 21.32504158 | 80.47476396 | 6.442475375 | 185.4974732 | rice  |
| 23 | 76 | 40 | 43 | 25.15745531 | 83.11713476 | 5.070175667 | 231.3843163 | rice  |
| 24 | 67 | 59 | 41 | 21.94766735 | 80.97384195 | 6.012632591 | 213.3560921 | rice  |
| 25 | 83 | 41 | 43 | 21.0525355  | 82.67839517 | 6.254028451 | 233.1075816 | rice  |
| 26 | 98 | 47 | 37 | 23.48381344 | 81.33265073 | 7.375482851 | 224.0581164 | rice  |
| 27 | 66 | 53 | 41 | 25.0756354  | 80.52389148 | 7.778915154 | 257.0038865 | rice  |

## Activity 2.2: Importing the Libraries

The next step in the project is to import the required Python libraries for data processing, visualization, and Machine Learning. Libraries such as Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn provide the necessary functions to analyze agricultural data, build prediction models, and evaluate their performance. The imported libraries used in the project are shown below.

```
1) import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Interactive shell and plot formatting as per your project requirements
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'

# Set pandas options and plot styles
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 50)
plt.style.use('fivethirtyeight')
plt.rcParams['figure.figsize'] = (12, 8)
%matplotlib inline

# Machine Learning Libraries
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report
```

## Activity 2.3: Read the Dataset

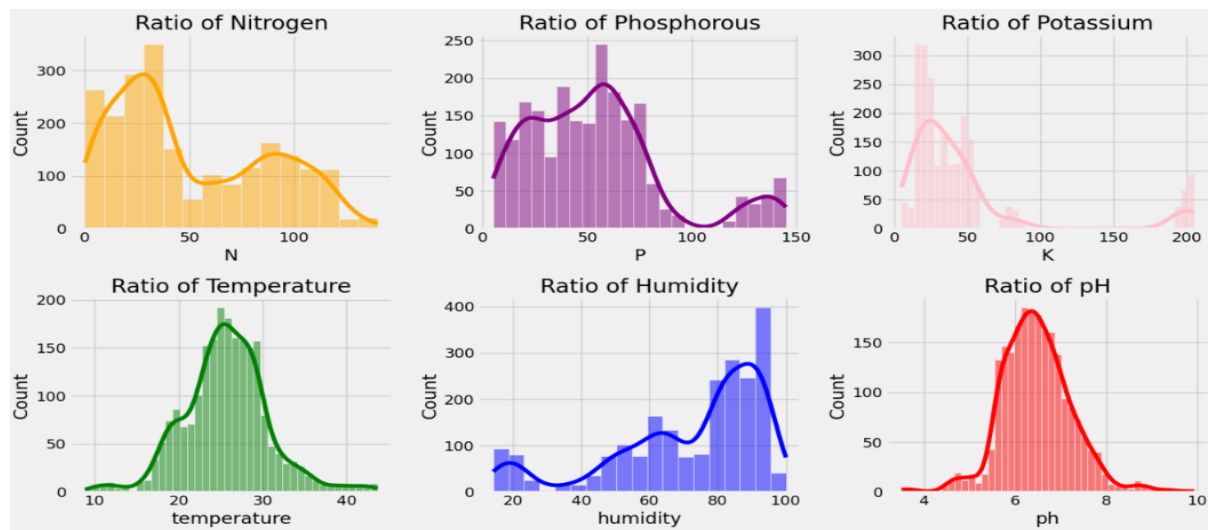
The downloaded Crop\_recommendation.csv dataset is loaded into the project using the read\_csv() function provided by the Pandas library. After loading the dataset, the head() function is used to display the first five records and verify that the data has been loaded successfully. The dataset contains important agricultural features such as Nitrogen (N), Phosphorous (P), Potassium (K), Temperature, Humidity, pH, Rainfall, and Crop Label, which are used for further data analysis and machine learning model development.

```
...
      N    P    K  temperature  humidity      ph  rainfall  label
0  90  42  43    20.879744  82.002744  6.502985  202.935536  rice
1  85  58  41    21.770462  80.319644  7.038096  226.655537  rice
2  60  55  44    23.004459  82.320763  7.840207  263.964248  rice
3  74  35  40    26.491096  80.158363  6.980401  242.864034  rice
4  78  42  42    20.130175  81.604873  7.628473  262.717340  rice
```



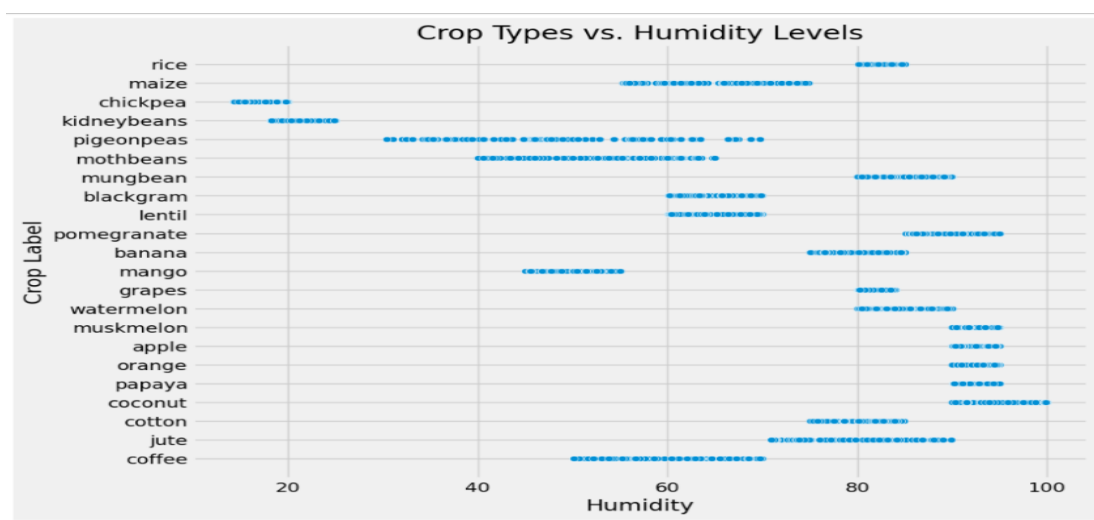
## Activity 2.4: Univariate Analysis:

Univariate Analysis is performed to examine the distribution of individual agricultural features in the crop recommendation dataset. Histogram plots are generated for parameters such as Nitrogen (N), Phosphorous (P), Potassium (K), Temperature, Humidity, pH, and Rainfall. This analysis helps identify the distribution, frequency, and variation of each feature, providing a better understanding of the dataset before model development.



## Activity 2.5: Bivariate Analysis

Bivariate Analysis is performed to examine the relationship between agricultural features and crop labels in the crop recommendation dataset. A scatter plot is used to visualize the relationship between Humidity and different Crop Labels. This analysis helps identify how humidity levels vary across different crops and provides insights into the influence of environmental conditions on crop recommendation. The observed relationships support better feature analysis and improve the performance of the machine learning model.



## Activity 2.6: Multivariate Analysis

Multivariate Analysis is performed to examine the combined relationship among multiple agricultural features and evaluate the performance of the crop recommendation model. A Confusion Matrix is generated to compare the actual crop labels with the predicted crop labels. This analysis helps measure the model's classification accuracy, identify correctly and incorrectly predicted crops, and evaluate the overall effectiveness of the machine learning model for crop recommendation.

```
[9]: # Visualize the confusion matrix to see where the model makes accurate predictions
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(15, 10))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=model.classes_, yticklabels=model.classes_)
plt.title('Confusion Matrix for Crop Prediction')
plt.xlabel('Predicted Crop')
plt.ylabel('Actual Crop')
plt.xticks(rotation=90)
plt.show()
```

```
[9]: <Figure size 1500x1000 with 0 Axes>
```

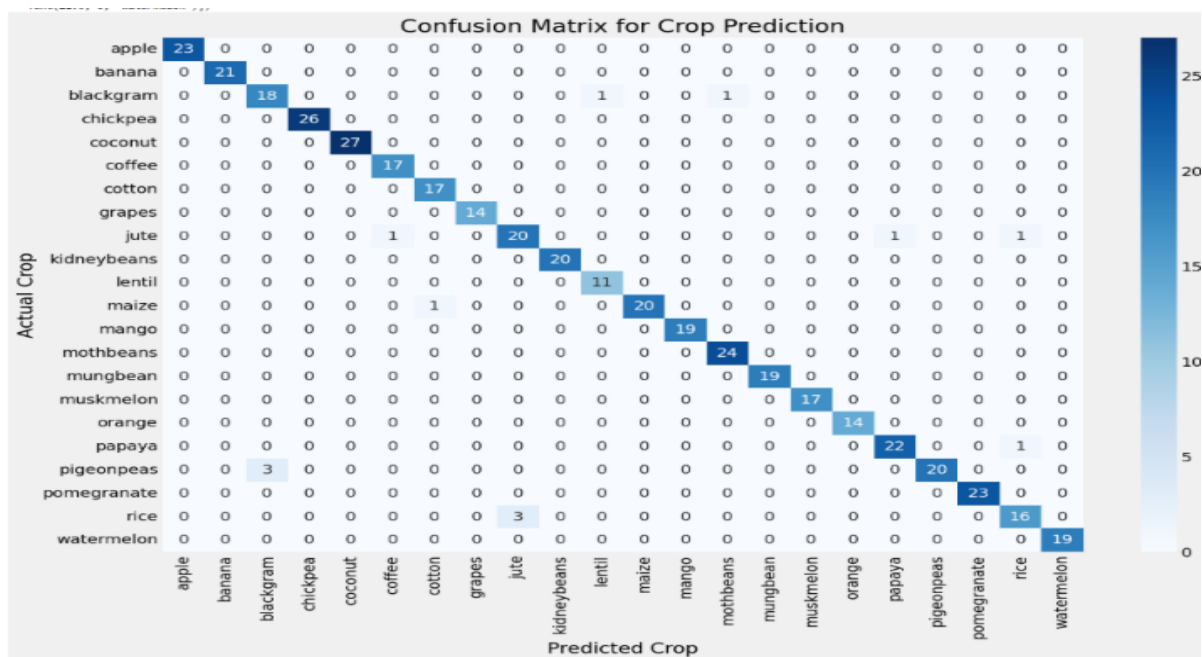
```
[9]: <Axes: >
```

```
[9]: Text(0.5, 1.0, 'Confusion Matrix for Crop Prediction')
```

```
[9]: Text(0.5, 40.58333333333333, 'Predicted Crop')
```

```
[9]: Text(90.58333333333333, 0.5, 'Actual Crop')
```

```
[9]: (array([[ 0.5,  1.5,  2.5,  3.5,  4.5,  5.5,  6.5,  7.5,  8.5,  9.5, 10.5,
          11.5, 12.5, 13.5, 14.5, 15.5, 16.5, 17.5, 18.5, 19.5, 20.5, 21.5]]),
 [Text(0.5, 0, 'apple'),
  Text(1.5, 0, 'banana'),
  Text(2.5, 0, 'blackgram'),
  Text(3.5, 0, 'chickpea'),
  Text(4.5, 0, 'coconut'),
  Text(5.5, 0, 'coffee'),
  Text(6.5, 0, 'cotton'),
  Text(7.5, 0, 'grapes'),
  Text(8.5, 0, 'jute'),
  Text(9.5, 0, 'kidneybeans'),
  Text(10.5, 0, 'lentil'),
  Text(11.5, 0, 'maize'),
  Text(12.5, 0, 'mango'),
  Text(13.5, 0, 'mothbeans'),
  Text(14.5, 0, 'mungbean'),
  Text(15.5, 0, 'muskmelon'),
  Text(16.5, 0, 'orange'),
  Text(17.5, 0, 'papaya'),
  Text(18.5, 0, 'pigeonpeas'),
  Text(19.5, 0, 'pomegranate'),
  Text(20.5, 0, 'rice'),
  Text(21.5, 0, 'watermelon']])
```



## Milestone 3: Data Pre-processing:

The dataset was pre-processed by checking for null values, removing outliers, extracting seasonal crop information, and splitting the data into training and testing sets. These steps ensured clean and reliable data for model building.

### Activity 3.1: Checking for Null Values

Before preprocessing the dataset, it is important to check whether it contains any missing or null values. This ensures that the dataset is complete and suitable for machine learning model development.

**Step 1 – Check for Null Values:** Use the `df.isnull().sum()` function to identify the number of missing values in each column of the dataset.

**Step 2 – Verify the Results:** The output confirms that all features contain zero null values, indicating that the dataset is complete and ready for further preprocessing.

```
df.isnull().sum()
```

```
N      0
P      0
K      0
temperature  0
humidity    0
ph          0
rainfall    0
label       0
dtype: int64
```

### Activity 3.2: Handling Outliers

Before training the Machine Learning model, it is important to identify and remove outliers from the dataset. Outliers are values that are much higher or lower than the normal range of data. These values may reduce the accuracy of the model and affect the prediction results. The Interquartile Range (IQR) method was used to detect and remove outliers from the crop recommendation dataset.

**Step 1:** Calculate the First Quartile (Q1), Third Quartile (Q3), and Interquartile Range (IQR).

**Step 2:** Find the lower and upper limits using the IQR method .

**Step 3:** Remove the values that are outside these limits and use the cleaned dataset for further processing.

```
# 2. Data Pre-processing: Handling Outliers
# =====
# Calculating Interquartile Range (IQR) for the phosphorous column
Q1 = df['phosphorous'].quantile(0.25)
Q3 = df['phosphorous'].quantile(0.75)
IQR = Q3 - Q1

# Defining boundaries
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Filtering the dataframe
df_filtered = df[(df['phosphorous'] >= lower_bound) & (df['phosphorous'] <= upper_bound)]
print(f"Data shape after removing outliers: {df_filtered.shape}\n")
```

```
Data shape after removing outliers: (2062, 8)
```

### Activity 3.3: Extracting Seasonal Crops

In this activity, the crop recommendation dataset was analyzed to identify crops suitable for different seasons. The dataset was filtered using temperature, humidity, and rainfall conditions to classify crops into summer, winter, and rainy seasons. This helps farmers understand which crops are suitable under different climatic conditions.

**Step 1: Apply Seasonal Conditions:** The filtered dataset was used to apply conditions based on temperature, humidity, and rainfall. The code extracts crops suitable for summer, winter, and rainy seasons.

```
# =====  
# 3. Extracting Seasonal Crops  
# =====  
print("--- Seasonal Crop Recommendations ---")  
summer_crops = df_filtered[(df_filtered['temperature'] > 30) & (df_filtered['humidity'] > 50)][['label']].unique()  
print(f"Summer crops: {summer_crops}")  
  
winter_crops = df_filtered[(df_filtered['temperature'] < 20) & (df_filtered['humidity'] > 30)][['label']].unique()  
print(f"Winter crops: {winter_crops}")  
  
rainy_crops = df_filtered[(df_filtered['rainfall'] > 200) & (df_filtered['humidity'] > 50)][['label']].unique()  
print(f"Rainy crops: {rainy_crops}\n")
```

**Step 2: Display Seasonal Crop Recommendations:** The extracted crop names for each season were displayed. This provides a list of crops suitable for summer, winter, and rainy seasons based on the given environmental conditions.

```
--- Seasonal Crop Recommendations ---  
Summer crops: ['pigeonpeas' 'mothbeans' 'blackgram' 'mango' 'grapes' 'orange' 'papaya']  
Winter crops: ['maize' 'pigeonpeas' 'lentil' 'pomegranate' 'grapes' 'orange']  
Rainy crops: ['rice' 'papaya' 'coconut']
```

### Activity 3.4: Splitting Data into Train and Test Sets

After completing data preprocessing, the dataset was prepared for Machine Learning model training by splitting it into training and testing datasets. The input features (X) and target variable (y) were separated. The `train_test_split()` function from Scikit-learn was used to divide the dataset into 80% training data and 20% testing data. This allows the model to learn from the training data and evaluate its performance using the testing data.

**Step 1: Split the Dataset:** The input features (X) and target variable (y) were separated from the dataset. The `train_test_split()` function was then used to divide the dataset into training and testing sets using an 80:20 ratio.

```
# =====  
# 4. Splitting Data into Train and Test Sets  
# =====  
# Separating features (X) and target label (y)  
X = df_filtered.drop('label', axis=1)  
y = df_filtered['label']  
  
# Splitting the data (80% training, 20% testing)  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)  
  
print("--- Data Split Shapes ---")  
print(f"X_train shape: {X_train.shape} | y_train shape: {y_train.shape}")  
print(f"X_test shape: {X_test.shape} | y_test shape: {y_test.shape}\n")
```

**Step 2: Verify the Data Split:** The shapes of the training and testing datasets were displayed to confirm that the data was divided correctly before model building.

```
--- Data Split Shapes ---  
X_train shape: (1649, 7) | y_train shape: (1649,)  
X_test shape: (413, 7) | y_test shape: (413,)
```

## Milestone 4: Model Development:

The pre-processed dataset was used to build a Machine Learning model for crop recommendation. K-Means Clustering was applied to group similar agricultural data, and Logistic Regression was used to predict suitable crops. The model was evaluated using classification metrics, and the trained model was saved for future predictions.

### Activity 4.1: K-Means Clustering

K-Means Clustering was applied to group similar agricultural conditions based on soil nutrients and environmental features in the crop recommendation dataset. The Elbow Method was first used to determine the optimal number of clusters by calculating the Within-Cluster Sum of Squares (WCSS) for different cluster values. The Elbow Graph indicated the most suitable number of clusters for the dataset. After selecting the optimal cluster value, the K-Means algorithm was trained to assign each data sample to a cluster. The generated clusters helped identify hidden patterns among crops with similar agricultural characteristics, providing valuable insights for further analysis and improving the overall crop recommendation process.

**Step 1:** Import the required libraries and initialize the K-Means Clustering algorithm.

```
import os
# This line fixes the memory leak warning by telling Windows how many threads to use
os.environ["OMP_NUM_THREADS"] = "9"

import pandas as pd
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# ... [Rest of your K-Means code from Step 2] ...
```

**Step 2:** Apply the Elbow Method by calculating WCSS values for different cluster numbers.

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# 1. Apply the Elbow Method to calculate WCSS
wcss = []
for i in range(1, 11):
    # n_init='auto' is the modern standard, replacing n_init=10
    kmeans = KMeans(n_clusters=i, init='k-means++', max_iter=300, n_init=10, random_state=0)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

# 2. Plot the Elbow Graph
plt.figure(figsize=(10, 4))
plt.plot(range(1, 11), wcss, marker='o', color='b')
plt.title('The Elbow Method', fontsize=20)
plt.xlabel('No of clusters')
plt.ylabel('WCSS')
plt.show()

# 3. Train the K-Means model with 4 clusters (as identified in your graph)
kmeans = KMeans(n_clusters=4, init='k-means++', max_iter=300, n_init=10, random_state=0)
y_kmeans = kmeans.fit_predict(X)

# 4. View which crops fall into which cluster
z = pd.concat([df['label'], pd.DataFrame(y_kmeans, columns=['cluster'])], axis=1)
print("Crops in First cluster:\n", z[z['cluster']==0]['label'].unique())
print("Crops in Second cluster:\n", z[z['cluster']==1]['label'].unique())
```

**Step 3:** Execute the K-Means algorithm for different cluster values and generate the clustering results.

```
KMeans(n_clusters=1, n_init=10, random_state=0)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.



KMeans

[?Documentation for KMeansFitted](#)

► Parameters

```
KMeans(n_clusters=2, n_init=10, random_state=0)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.



KMeans

[?Documentation for KMeansFitted](#)

► Parameters

```
KMeans(n_clusters=3, n_init=10, random_state=0)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

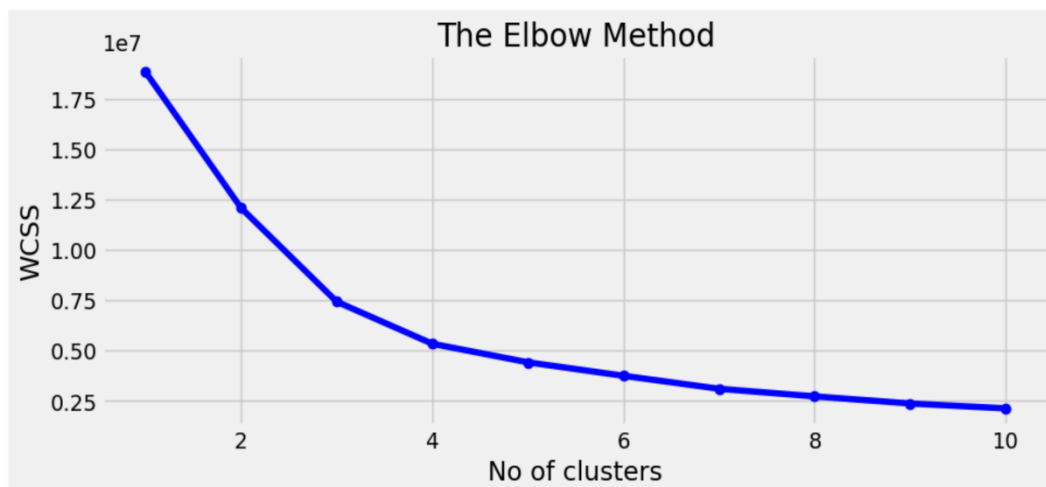


KMeans

[?Documentation for KMeansFitted](#)

► Parameters

**Step 4:** Plot the Elbow Graph and identify the optimal number of clusters. Display the grouped crop clusters.



Crops in First cluster:

```
['grapes' 'apple']
```

Crops in Second cluster:

```
['maize' 'chickpea' 'kidneybeans' 'pigeonpeas' 'mothbeans' 'mungbean'
'blackgram' 'lentil' 'pomegranate' 'mango' 'orange' 'papaya' 'coconut']
```

## Activity 4.2: Logistic Regression Model Training

The Logistic Regression algorithm was implemented to predict the most suitable crop based on the given agricultural parameters. The model was initialized and trained using the scaled training dataset after preprocessing. Once the training process was completed, the trained Logistic Regression model was successfully generated and made ready for prediction. This model learns the relationship between the input features and crop labels, enabling accurate crop recommendations based on soil nutrients and environmental conditions.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

# 1. Initialize the model
model = LogisticRegression()

# 2. Train the model using the SCALED training data
model.fit(X_train_scaled, y_train)

# 3. Make predictions using the SCALED test data
y_pred = model.predict(X_test_scaled)

# 4. Evaluate the model's performance metrics
cr = classification_report(y_test, y_pred)
print("Classification Report:\n", cr)

LogisticRegression()
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.
LogisticRegression
?Documentation for LogisticRegressionFitted
► Parameters
```

## Activity 4.3: Evaluate Model Performance

The performance of the trained Logistic Regression model was evaluated using the Classification Report, which provides important evaluation metrics such as Precision, Recall, F1-Score, and Accuracy. These metrics help measure the model's prediction performance for each crop class and assess its overall classification capability. The evaluation results indicate that the model achieved high prediction accuracy and reliable performance, making it suitable for recommending the most appropriate crop based on agricultural conditions.

| Classification Report: |           |        |          |         |
|------------------------|-----------|--------|----------|---------|
|                        | precision | recall | f1-score | support |
| apple                  | 1.00      | 1.00   | 1.00     | 23      |
| banana                 | 1.00      | 1.00   | 1.00     | 21      |
| blackgram              | 0.90      | 0.95   | 0.93     | 20      |
| chickpea               | 1.00      | 1.00   | 1.00     | 26      |
| coconut                | 1.00      | 1.00   | 1.00     | 27      |
| coffee                 | 0.94      | 1.00   | 0.97     | 17      |
| cotton                 | 0.94      | 1.00   | 0.97     | 17      |
| grapes                 | 1.00      | 1.00   | 1.00     | 14      |
| jute                   | 0.83      | 0.87   | 0.85     | 23      |
| kidneybeans            | 0.95      | 0.95   | 0.95     | 20      |
| lentil                 | 0.85      | 1.00   | 0.92     | 11      |
| maize                  | 1.00      | 0.95   | 0.98     | 21      |
| mango                  | 1.00      | 1.00   | 1.00     | 19      |
| mothbeans              | 1.00      | 0.92   | 0.96     | 24      |
| mungbean               | 1.00      | 1.00   | 1.00     | 19      |
| muskmelon              | 1.00      | 1.00   | 1.00     | 17      |
| orange                 | 1.00      | 1.00   | 1.00     | 14      |
| papaya                 | 0.96      | 0.96   | 0.96     | 23      |
| pigeonpeas             | 0.91      | 0.87   | 0.89     | 23      |
| pomegranate            | 1.00      | 1.00   | 1.00     | 23      |
| rice                   | 0.88      | 0.79   | 0.83     | 19      |
| watermelon             | 1.00      | 1.00   | 1.00     | 19      |
| accuracy               |           |        | 0.96     | 440     |
| macro avg              | 0.96      | 0.97   | 0.96     | 440     |
| weighted avg           | 0.96      | 0.96   | 0.96     | 440     |



## Activity 4.4: Save the Trained Model and Predict Crop

The trained Logistic Regression model and the Standard Scaler were saved using the Pickle library so that they can be reused for future predictions without retraining. The saved model and scaler were then loaded to accept new user input, preprocess the input data, and predict the most suitable crop based on the given agricultural parameters. The output confirmed that the model was successfully saved and generated the recommended crop accurately.

**Step 1:** Save the trained Logistic Regression model and Standard Scaler using the Pickle library.

```
import pickle
import numpy as np
import warnings
warnings.filterwarnings("ignore") # Hides standard sklearn warnings

# 1. Save both the model and the scaler to your computer
pickle.dump(model, open('logistic_model.pkl', 'wb'))
pickle.dump(scaler, open('scaler.pkl', 'wb'))
print("Model and Scaler successfully saved!")

# -----
# HOW TO USE THE SAVED MODEL FOR A NEW PREDICTION:
# -----

# 2. Load the saved model and scaler
loaded_model = pickle.load(open('logistic_model.pkl', 'rb'))
loaded_scaler = pickle.load(open('scaler.pkl', 'rb'))

# 3. Accept user input features in this order:
# [Nitrogen, Phosphorus, Potassium, temperature, humidity, ph, rainfall]
user_input = np.array([[90, 42, 43, 20.8, 82.0, 6.5, 202.9]])

# 4. Scale the user's input using the loaded scaler (crucial for accurate predictions)
scaled_input = loaded_scaler.transform(user_input)

# 5. Make the final prediction
prediction = loaded_model.predict(scaled_input)
print(f"\nThe suggested crop for the given climatic condition is: '{prediction[0]}'")
```

**Step 2:** Load the saved model and scaler, provide user input, and predict the recommended crop.

Model and Scaler successfully saved!

The suggested crop for the given climatic condition is: 'rice'

## Milestone 5: Application Development:

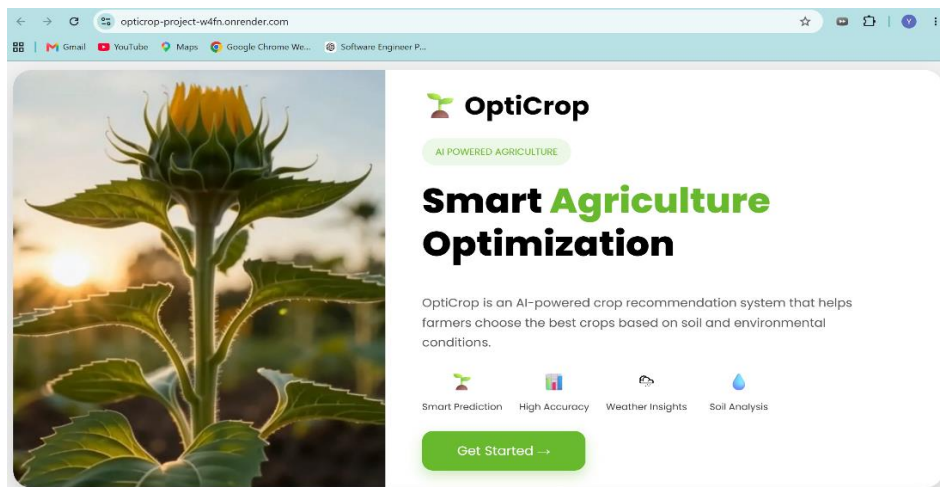
In this milestone, the OptiCrop web application was developed by creating responsive web pages, implementing the Flask-based Python backend, and integrating the trained machine learning model. The application was then tested to ensure that it accepts user inputs and provides accurate crop recommendations based on soil and environmental parameters.

### Activity 5.1: Building HTML Pages

In this activity, a responsive and user-friendly web interface was developed for the OptiCrop application using HTML, CSS, and Bootstrap. The interface consists of a homepage and a crop prediction page where users can enter soil and environmental parameters. Bootstrap was used to create a responsive layout, while CSS was used to improve the appearance and user experience of the application.

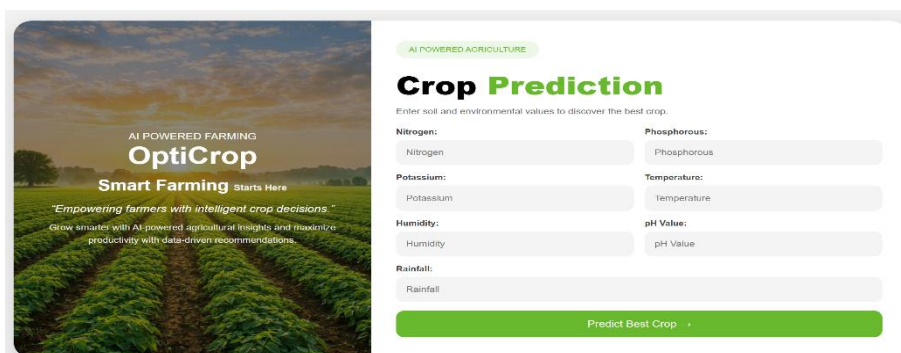
#### Step 1: Design the Homepage

Designed the OptiCrop homepage using HTML, CSS, and Bootstrap with an attractive layout, project title, and Get Started button.



#### Step 2: Create the Crop Prediction Page

Developed a responsive page containing a form to collect soil and environmental parameters from the user.



### Step 3: Apply Responsive Design

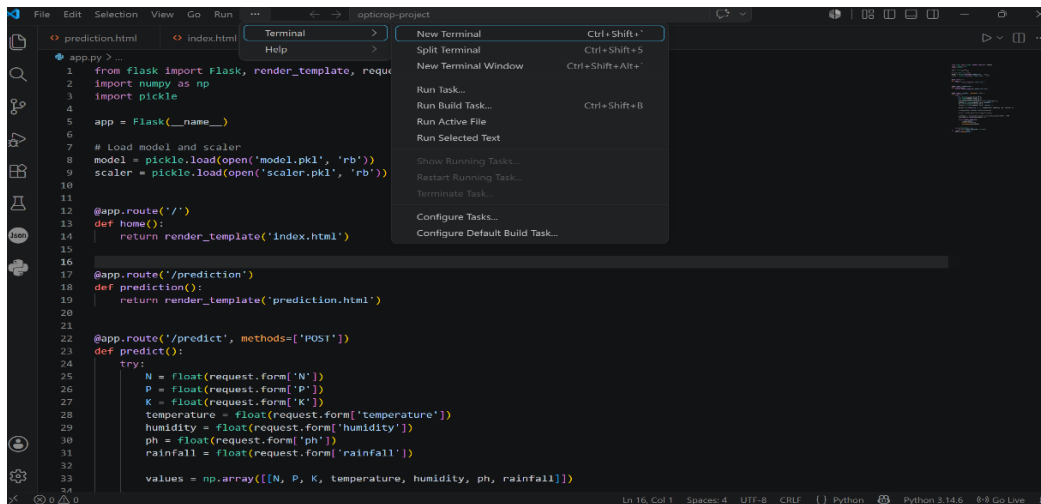
Used Bootstrap and CSS to make the interface responsive, user-friendly, and compatible with different screen sizes.

## Activity 5.2: Build Python Backend Code

The backend of the OptiCrop application was developed using the Flask framework. The backend was connected with the trained machine learning model to process user inputs and generate crop recommendations.

### Step 1: Develop the Flask Backend

Created the Flask application and implemented the backend logic for handling user requests and predictions.



```

1 from flask import Flask, render_template, request
2 import numpy as np
3 import pickle
4
5 app = Flask(__name__)
6
7 # Load model and scaler
8 model = pickle.load(open('model.pkl', 'rb'))
9 scaler = pickle.load(open('scaler.pkl', 'rb'))
10
11
12 @app.route('/')
13 def home():
14     return render_template('index.html')
15
16
17 @app.route('/prediction')
18 def prediction():
19     return render_template('prediction.html')
20
21
22 @app.route('/predict', methods=['POST'])
23 def predict():
24     try:
25         N = float(request.form['N'])
26         P = float(request.form['P'])
27         K = float(request.form['K'])
28         temperature = float(request.form['temperature'])
29         humidity = float(request.form['humidity'])
30         ph = float(request.form['ph'])
31         rainfall = float(request.form['rainfall'])
32         values = np.array([[N, P, K, temperature, humidity, ph, rainfall]])

```

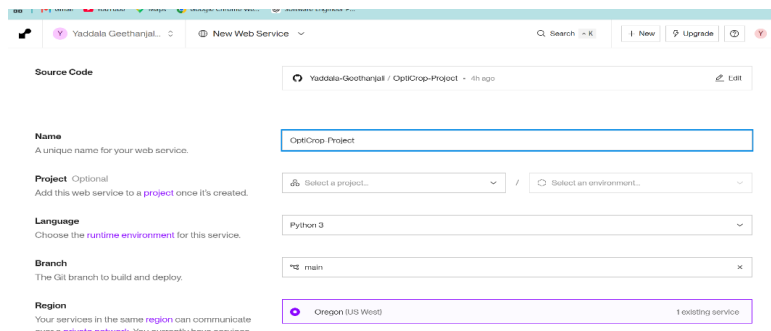
### Step 2: Run the Flask Application

Executed the application using the python app.py command to start the local server.

```
PS D:\OneDrive\Documents\opticrop-project> python app.py
```

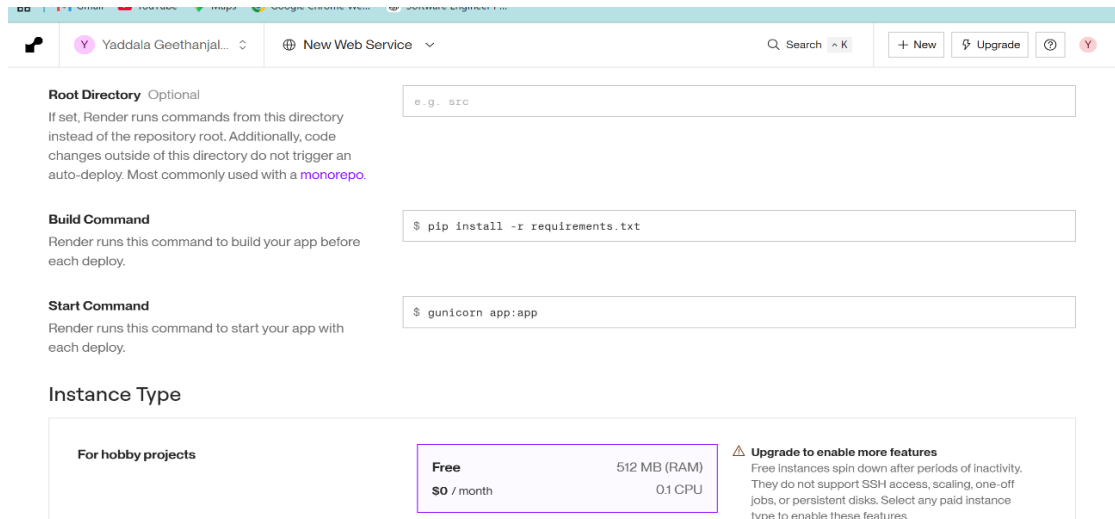
### Step 3: Create the Web Service

Created a new web service in Render and connected the GitHub repository for deployment.



## Step 4: Configure Deployment Settings

Configured the build command, start command, and deployment settings in Render to host the application.



**Root Directory** Optional  
If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a [monorepo](#).

**Build Command**  
Render runs this command to build your app before each deploy.

**Start Command**  
Render runs this command to start your app with each deploy.

**Instance Type**

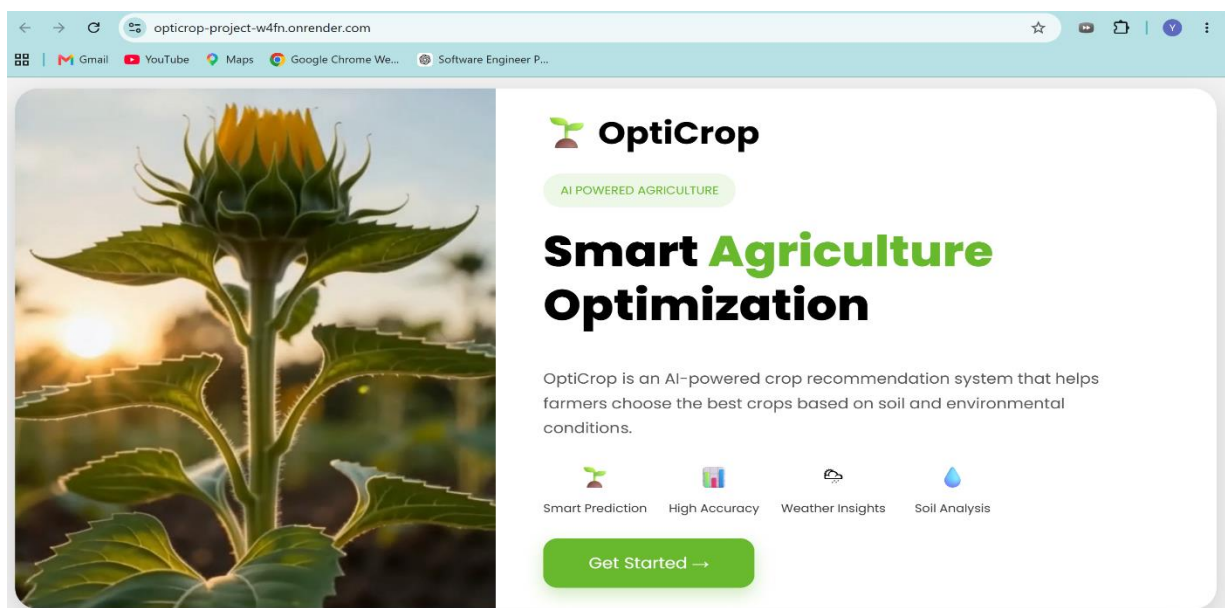
| For hobby projects | Free        | 512 MB (RAM) | 0.1 CPU |
|--------------------|-------------|--------------|---------|
|                    | \$0 / month |              |         |

**Upgrade to enable more features**  
Free instances spin down after periods of inactivity. They do not support SSH access, scaling, one-off jobs, or persistent disks. Select any paid instance type to enable these features.

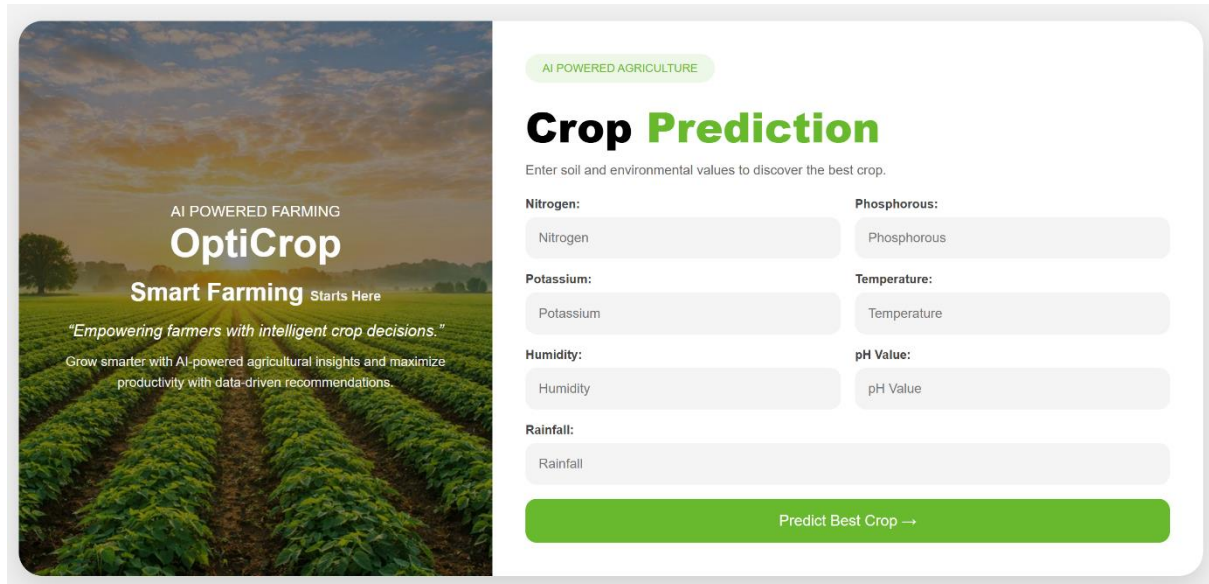
## Activity 5.3: Run the Application

In this activity, the OptiCrop application was executed to provide crop recommendations based on user-provided soil and environmental parameters. Users access the application through the homepage, navigate to the crop prediction page, enter the required values, and receive the recommended crop along with the confidence score. The prediction results are displayed in a clear and user-friendly format.

### Step 1: Launch the OptiCrop application and open the homepage.

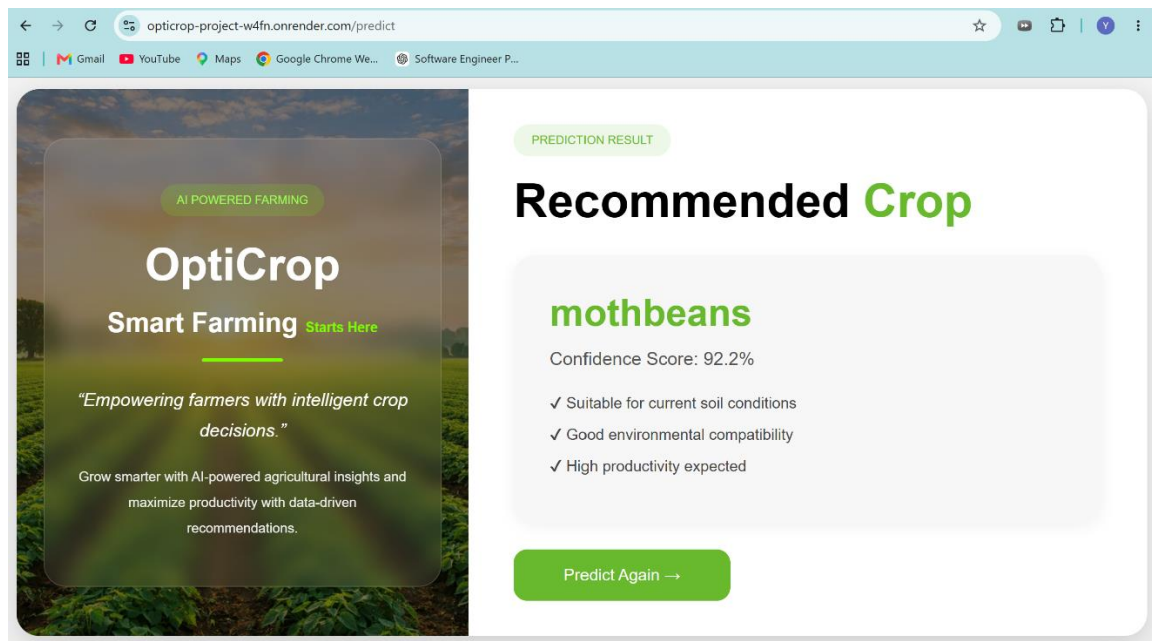


**Step 2:** Navigate to the Crop Prediction page and enter the required soil and environmental parameters.



The screenshot shows the 'Crop Prediction' page of the OptiCrop application. On the left, there is a banner image of a field with the text 'AI POWERED FARMING OptiCrop Smart Farming Starts Here' and a quote: 'Empowering farmers with intelligent crop decisions.' Below this, it says 'Grow smarter with AI-powered agricultural insights and maximize productivity with data-driven recommendations.' On the right, there is a form titled 'Crop Prediction' with the instruction 'Enter soil and environmental values to discover the best crop.' The form contains six input fields: Nitrogen, Phosphorous, Potassium, Temperature, Humidity, and pH Value. Below these fields is a 'Rainfall' label and a corresponding input field. At the bottom of the form is a green button labeled 'Predict Best Crop →'.

**Step 3:** Click the Predict Best Crop button to generate the prediction. The application displays the recommended crop along with the confidence score.



The screenshot shows the 'Prediction Result' page of the OptiCrop application. On the left, there is a banner image of a field with the text 'AI POWERED FARMING OptiCrop Smart Farming Starts Here' and a quote: 'Empowering farmers with intelligent crop decisions.' Below this, it says 'Grow smarter with AI-powered agricultural insights and maximize productivity with data-driven recommendations.' On the right, there is a section titled 'Recommended Crop' with the text 'mothbeans' in green. Below this, it says 'Confidence Score: 92.2%' and lists three bullet points: '✓ Suitable for current soil conditions', '✓ Good environmental compatibility', and '✓ High productivity expected'. At the bottom of the section is a green button labeled 'Predict Again →'.

## **Milestone 6: Conclusion:**

The OptiCrop: Smart Agricultural Production Optimization Engine was successfully developed to recommend the most suitable crop based on soil and environmental parameters. The project included data preprocessing, machine learning model development, web application development, and deployment using the Flask framework. A responsive user interface was designed to allow users to easily enter input values and receive accurate crop recommendations. The application demonstrates the practical use of machine learning in agriculture and helps farmers make informed decisions to improve crop productivity and resource utilization.